최신 AI 모델을 이용한
Flow 기반 네트워크 이상탐지

제출일	2025.11.11	**소속학부**	소프트웨어학부
과목명	캡스톤디자인프로젝트	**조명**	회의하러왔습니다
담당교수	이상준	**조원**	2020xxxx 김○서 2021xxxx 송○용 2021xxxx 안○엽 2021xxxx 이수아
지도교수	주경호		

- 목 차 -

1. 프로젝트 소개	.....	3
2. 기존 시스템 분석	.....	3
3. 특징 및 기능	.....	4
4. 인터페이스	.....	5
5. 소프트웨어 요구사항	.....	6
6. 시스템 대블록 설계	.....	7
7. 구성도	.....	8
8. 유즈케이스 다이어그램	.....	10
9. 클래스 다이어그램	.....	10
10. 시퀀셜 다이어그램	.....	11
11. 알고리즘 구성	.....	11
12. 데이터베이스 테이블 구조	.....	15
13. 시나리오	.....	15
14. 구현결과	.....	16
15. 역할 분담 및 개발일정	.....	24
16. 관련 기술 및 개념	.....	22
17. 기대효과	.....	31
18. 개발도구 및 개발환경	.....	32
19. 참고 문헌	.....	32

1. 프로젝트 소개

본 프로젝트는 네트워크 환경에서 발생하는 Flow 로그 데이터를 기반으로 이상트래픽을 탐지하는 AI 시스템을 개발하는 것을 목표로 한다.

Flow 데이터는 각 패킷이 아닌 세션 단위의 패킷 헤더 필드(메타데이터)를 집계한 정보로, 트래픽의 흐름을 효율적으로 요약할 수 있다. 이를 통해 실제 패킷 내용(payload)을 직접 분석하지 않더라도 네트워크 내의 비정상적인 트래픽을 효과적으로 파악할 수 있다. 특히, 기존의 시그니처 기반 탐지 시스템(예: Snort, Suricata)은 미리 정의된 규칙에 의존하기 때문에 새로운 형태의 공격(변형 트래픽)이나 정상 트래픽에 유사한 통신을 탐지하기 어렵다. 반면, 본 프로젝트에서는 AI 모델이 기존의 정상적인 트래픽 패턴을 학습하여 비정상적인 트래픽을 탐지하는 것이 목표이다.

학습에 사용하는 데이터셋은 'CICIDS2017', 'CSE-CIC-IDS2018' 등의 공공 데이터셋을 활용한다. 이때, 공공 데이터셋은 L3(네트워크) + L4(전송) 헤더 필드 기반의 Flow 데이터셋을 활용하며, 해당 데이터 셋을 다양한 머신러닝/딥러닝 모델 기반으로 실험하여 적절한 이상 데이터셋을 찾아내는 최적의 모델을 만드는 것을 목표로 한다.

2. 기존 시스템 분석

기존의 네트워크 침입탐지 시스템(NIDS)은 대부분 서명(Signature) 기반 탐지 방식을 사용한다. 이러한 시스템은 알려진 공격 패턴에는 효과적이지만, 미리 정의되지 않은 제로 데이 공격이나 변형된 형태의 비정상 트래픽은 탐지하지 못하는 한계를 가진다. 또한 패킷 단위 분석은 처리 비용이 높고, 실시간 대응에 어려움이 있다.

반면 본 프로젝트는 이러한 한계를 극복하기 위해 Flow 로그 기반 분석을 도입한다. Flow 로그는 트래픽을 세션 단위로 집계하여 효율적으로 처리하며 AI의 비지도 학습능력을 결합하여 기존방식으로는 탐지 불가능했던 비정규적/저속 통신 패턴까지 자동화된 예측 및 탐지 서비스를 제공한다.

이러한 접근법은 기존 NIDS인 Snort나 Suricata가 개별 패킷을 실시간으로 분석하여 규칙과 비교하는 것과 달리, AI모델이 학습한 정상 트래픽의 분포 기준선에서 벗어나는 이상행위를 포착하는데 중점을 둔다. 이를 통해 새로운 유형의 위협에 대한

대응 능력을 강화하고, 탐지 시스템의 효율성과 정확도를 높일 수 있다.

3. 특징 및 구상도

- **특징**

패킷 메세지(payload)를 포함하지 않는 Flow 헤더 필드 데이터를 분석하여 저장 및 연산 비용을 절감할 수 있다. 사용하고자 하는 모델은, 2~3가지 머신러닝/딥러닝 모델에 전처리한 Flow 데이터셋을 학습시킨 후, 가장 성능이 뛰어난 머신러닝/딥러닝 모델을 선정하여 결과를 도출하고자 한다. 이때, 성능은 단순히 정확도만이 아닌 FPR + Recall 트레이드오프(운영에 맞는 임계값) 기준으로 선정한다. 이때 간략한 진행 순서는 다음과 같다. 3개의 머신러닝/딥러닝 모델을 동일한 정상/비정상 Flow 데이터셋을 통해 학습시켜 정상 트래픽의 복잡한 패턴과 통계적 특성을 파악한다 → 학습된 모델이 학습한 패턴과 비교하여 새로 들어온 Flow 데이터가 평균에서 벗어나는지(이상치)를 파악하고 발견 시 네트워크 공격으로 탐지한다 → 학습 시킨 머신러닝/딥러닝 모델 3가지를 평가기준(Accuracy + FPR + Recall 트레이드오프)을 통해 성능을 분석한다 → 평가기준의 결과를 분석하여 가장 적절한 모델을 선정한다.

- **진행 순서도**

① 네트워크 트래픽 수집 (NetFlow / VPC Flow / Zeek)

↓

② Flow 정규화 및 저장 (Raw Flow DB)

↓

③ 전처리 및 특징 추출

↓

④ AI 모델 학습 (LSTM, TCN-Autoencode 등)

↓

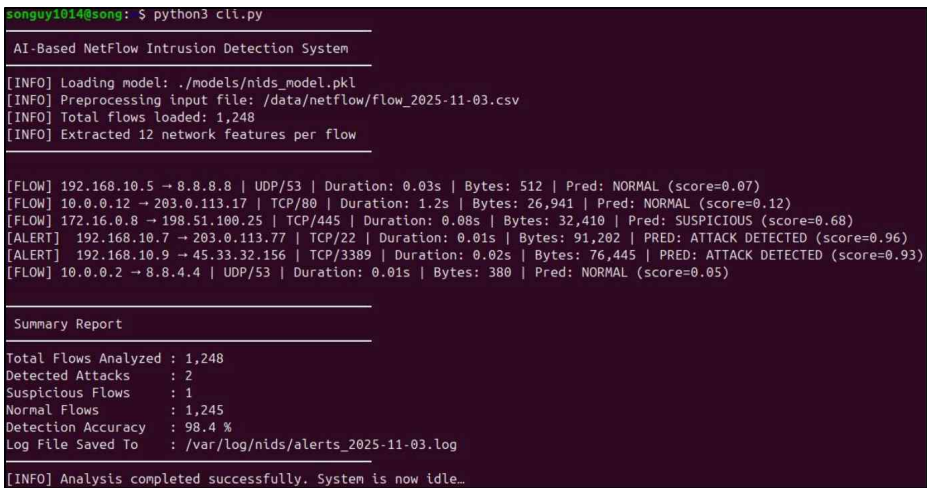
⑤ 성능 평가 및 모델 선정

↓

⑥ 실시간 이상행위 탐지 결과 반환

4. 인터페이스 (UI)

4-1. 상세 _ 웹 대시보드 기반 UI 화면



```
songuy1014@song: $ python3 cli.py
═══════════════════════════════════════════
 AI-Based NetFlow Intrusion Detection System
═══════════════════════════════════════════
[INFO] Loading model: ./models/nids_model.pkl
[INFO] Preprocessing input file: /data/netflow/flow_2025-11-03.csv
[INFO] Total flows loaded: 1,248
[INFO] Extracted 12 network features per flow
═══════════════════════════════════════════

[FLOW] 192.168.10.5 → 8.8.8.8 | UDP/53 | Duration: 0.03s | Bytes: 512 | Pred: NORMAL (score=0.07)
[FLOW] 10.0.0.12 → 203.0.113.17 | TCP/80 | Duration: 1.2s | Bytes: 26,941 | Pred: NORMAL (score=0.12)
[FLOW] 172.16.0.8 → 198.51.100.25 | TCP/445 | Duration: 0.08s | Bytes: 32,410 | Pred: SUSPICIOUS (score=0.68)
[ALERT]  192.168.10.7 → 203.0.113.77 | TCP/22 | Duration: 0.01s | Bytes: 91,202 | PRED: ATTACK DETECTED (score=0.96)
[ALERT]  192.168.10.9 → 45.33.32.156 | TCP/3389 | Duration: 0.02s | Bytes: 76,445 | PRED: ATTACK DETECTED (score=0.93)
[FLOW] 10.0.0.2 → 8.8.4.4 | UDP/53 | Duration: 0.01s | Bytes: 380 | Pred: NORMAL (score=0.05)

═══════════════════════════════════════════
 Summary Report
═══════════════════════════════════════════
Total Flows Analyzed : 1,248
Detected Attacks     : 2
Suspicious Flows     : 1
Normal Flows         : 1,245
Detection Accuracy   : 98.4 %
Log File Saved To    : /var/log/nids/alerts_2025-11-03.log
═══════════════════════════════════════════
[INFO] Analysis completed successfully. System is now idle…
```

4-2. 상세 _ CLI 기반 UI 화면

5. 소프트웨어 요구사항(Software Requirement)

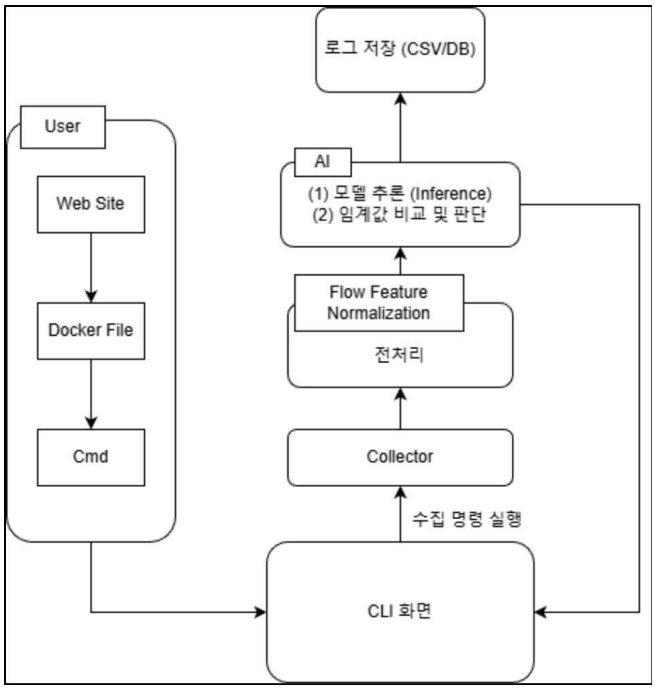
본 시스템은 네트워크 플로우(Flow) 데이터를 활용하여 이상 트래픽을 탐지하는 프로그램으로, 사용자의 네트워크 로그 데이터를 추출하고 이를 분석할 수 있도록 설계한다. 수집된 데이터는 전처리 과정을 거쳐 학습용/테스트용 데이터셋으로 분리되어 변환되며, 이를 기반으로 최신 AI 모델을 적용하여 정상 트래픽과 비정상 트래픽을 분류한다.

AI 모델은 네트워크 공격 유형에 대한 패턴을 학습하여 새로운 트래픽 데이터가 주어졌을 때 이상 여부를 판단한다. 이를 통해 기존 시그니처 기반 탐지의 한계를 보완하고, 알려지지 않은 공격에도 유연하게 대응 가능한 이상 탐지 기능을 제공한다.

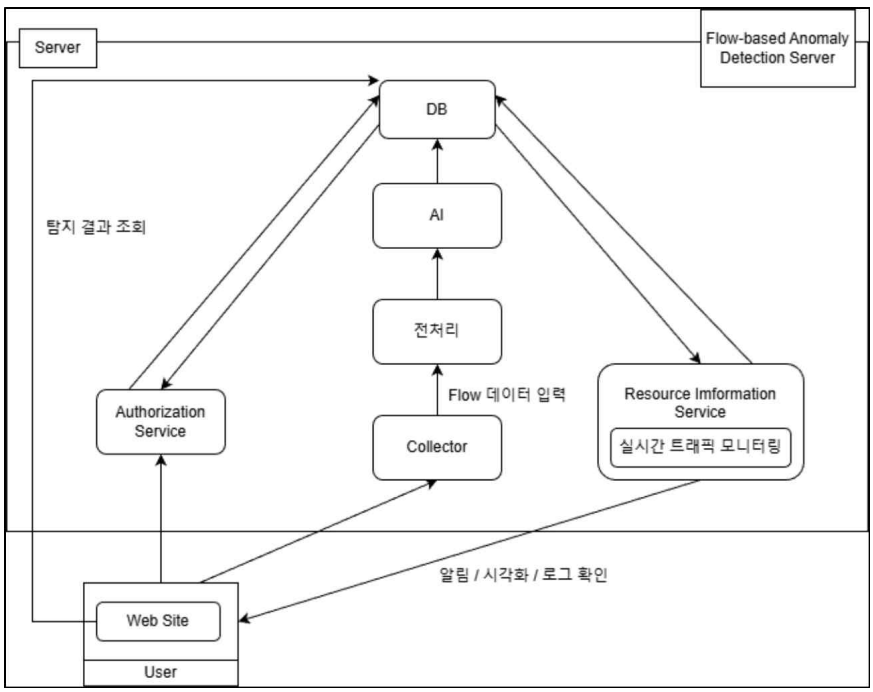
시스템은 학습 단계/테스트 단계, 탐지 단계로 구분된다. 학습 단계에서는 공개 데이터셋을 기반으로 모델을 학습하고, 테스트 단계에서는 탐지 성능 평가를 통해 모델의 정확도와 적절한 임계값을 검증하여 탐지 기준을 최적화한다. 마지막 탐지 단계에서는 학습된 모델을 이용하여 사용자의 네트워크 Flow Data Collector의 실시간 또는 저장된 트래픽 데이터를 분석하여 사용자에게 결과를 출력한다.

본 프로젝트는 Flow 기반 네트워크 공격 탐지의 효율성을 향상시키기 위해 최신 인공지능 기술을 적용하고, 다양한 공격 유형에 대한 탐지 성능을 비교·분석함으로써 AI가 탐지한 적절한 결과를 유저에게 제공하는 것을 목표로 한다.

6. 시스템 대블록 설계

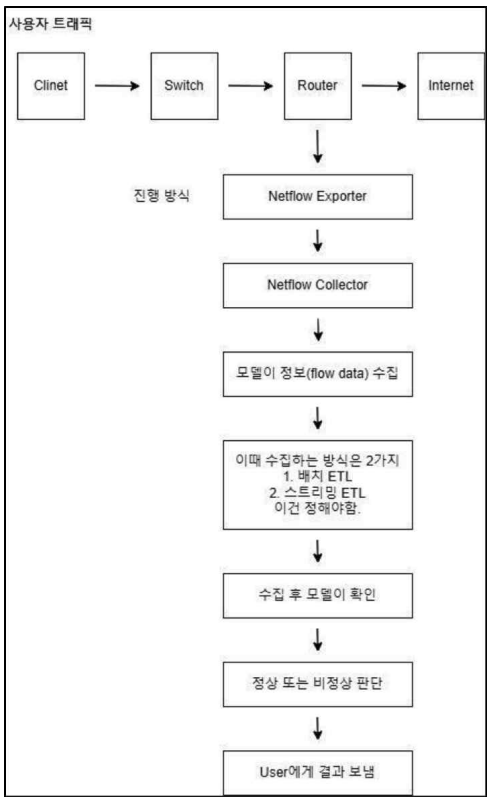


6-1. 상세 _ CLI 기반 시스템 구성도 (로컬 학습 구조)

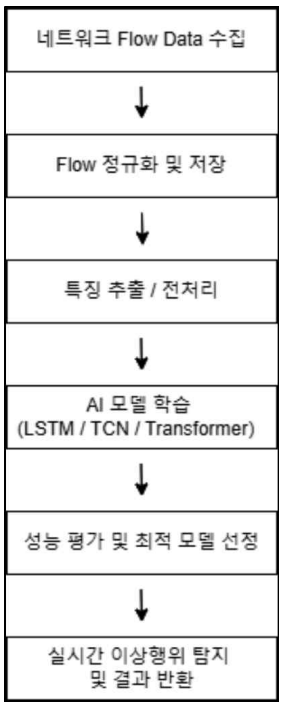


6-2. 상세 _ API 기반 시스템 구성도 (서버 배포 구조)

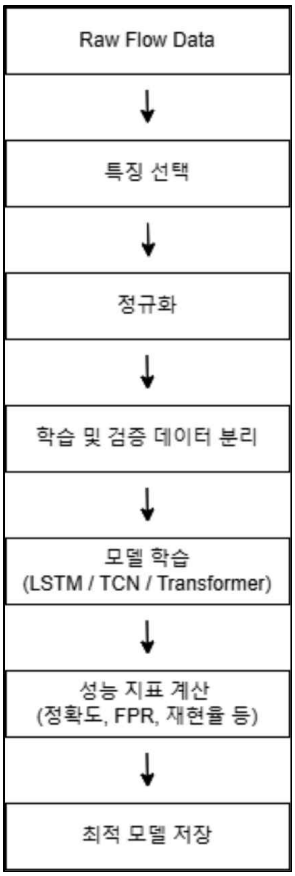
7. 구성도 (Flow Chart)

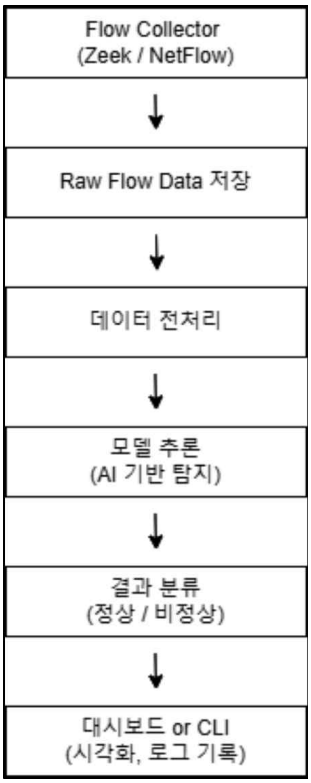


7-1. 상세 _ 시스템 전체 흐름도



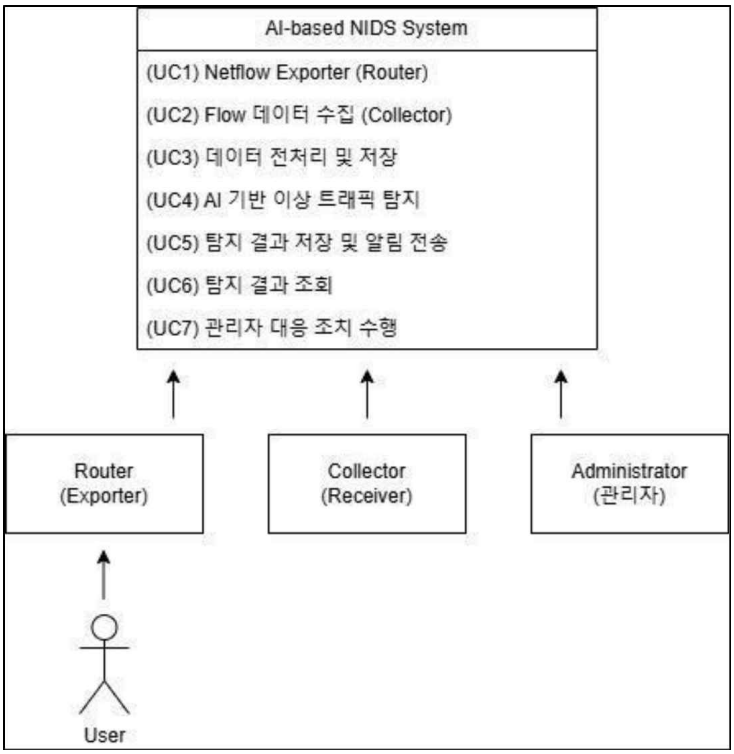
7-2. 상세 _ AI 학습 단계 흐름도



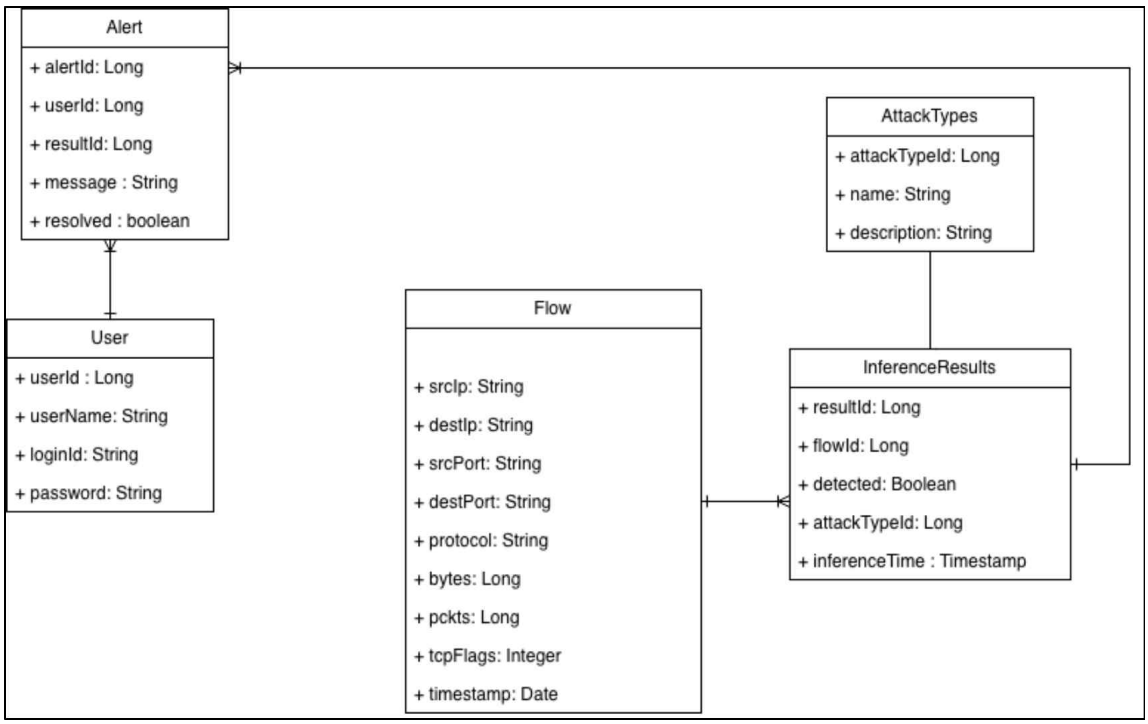


7-3. 상세 _ 실시간 탐지 단계 흐름도

8. 유즈케이스 다이어그램 (Usecase Diagram)

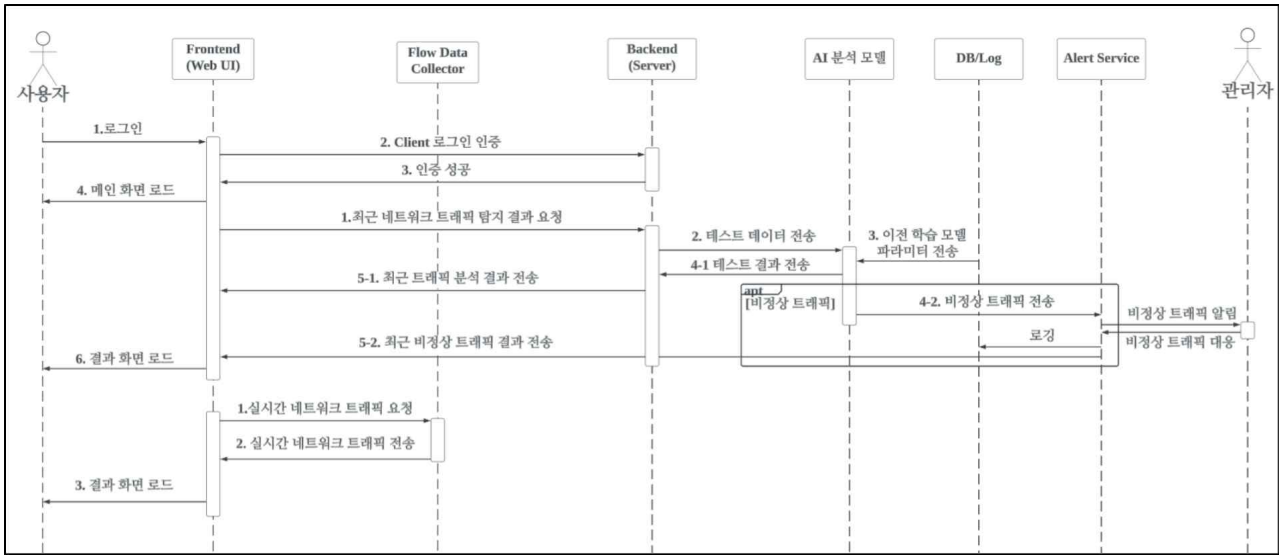


9. 클래스 다이어그램 (Class Diagram)



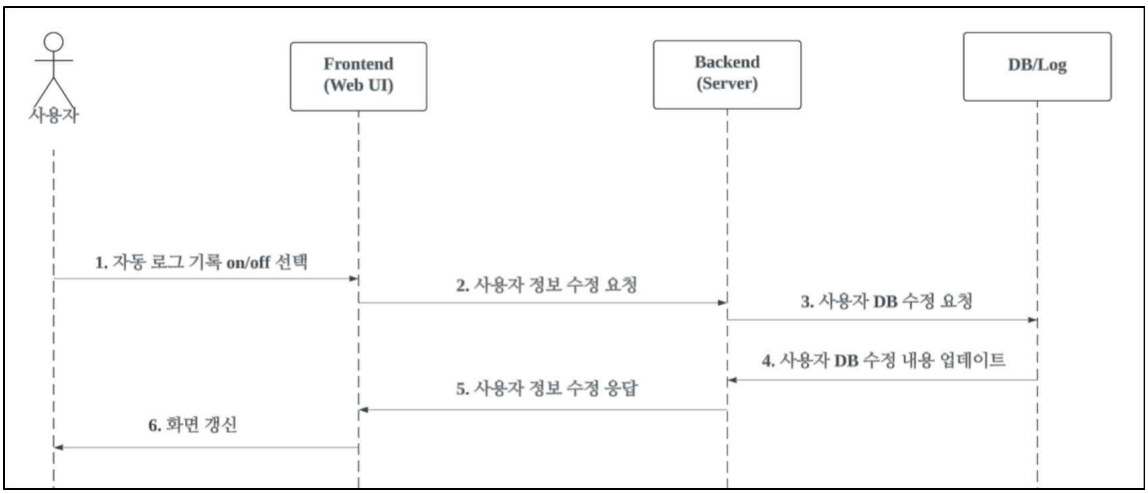
10. 시퀀셜 다이어그램 (Sequential Diagram)

10.1. 실시간 네트워크 트래픽 현상 및 최근 이상 탐지



10.2. 사용자 UI 설정

- 자동 로그 기록 여부, 관리자 실시간 알림 여부, 알림 문구, 민감도 조정, 자동화 규칙 설정 등



11. 알고리즘 구성 (Algorithm)

11.1. 입력 및 출력 정의

- 입력(Input) : Netflow / Zeek 기반 세션 단위 Flow 로그

  → (src/dst IP, Port, Protocol, Duration, Bytes, Packets, TCP Flags 등)

- 출력(Output) : Flow별

  → 이상 점수

  → 정상 / 비정상 판정 결과

  → 탐지 시간, 공격 유형, 위험도 등 메타데이터

11.2 데이터 전처리 및 특징 추출

(1) 정규화(Normalization)

- StandardScaler 또는 RobustScaler 적용
- bytes, duration 등은 log 스케일 변환

(2) 파생 특징 생성

- bytes/sec, pkts/sec, bytes/pkt, 양방향 비율, TCP Flag 비율 등

(3) 결측치 및 이상값 처리

- 결측값은 중앙값 대치, 이상치는 퍼센타일(0.1~99.9%) 범위로 클리핑

(4) 특징 선택(Feature Selection)

- 상관계수, Variance Threshold, Mutual Info 기반 중요 Feature만 남김

11.3 모델 구조 (Model Architecture)

본 시스템은 Autoencoder 기반 이상 탐지 모델을 사용한다. Encoder–Decoder 구조로 입력 데이터 X를 재구성하고, 입력값과 복원값 간의 차이(‖X - X̂‖)를 재구성 오차(Reconstruction Error)로 계산한다. 이 오차가 일정 임계값을 초과하면 비정상으로 판단한다.

(1) 주요 모델 후보

- LSTM-Autoencoder : 시퀀스 종속성을 학습, 저속 C2 및 주기성 트래픽 탐지에 강함
- TCN-Autoencoder : 합성곱 기반 병렬 처리로 빠르고, 장기 의존성 포착력 높음

- Seq-Transformer (경량형) : Attention 기반으로 병렬성 및 효율성 우수

(2) 비교·보조 모델

- Isolation Forest, One-Class SVM과 비교 실험을 수행해 Autoencoder 성능 검증
- 필요 시 앙상블 구성으로 FP(오탐) 감소

11.4 학습 및 검증 (Training & Validation)

(1) 데이터 분할 : Train / Validation / Test (시간 순서 유지)

(2) 학습 방식 : 정상 데이터만으로 비지도 학습 (MSE, MAE 기반 Reconstruction Loss)

(3) 평가 지표 : Accuracy, Recall, FPR, PR-AUC, ROC-AUC

(4) 모델 선정 기준 : 성능(FPR+Recall) + 추론속도 + 메모리 효율

(5) 최종 모델 : LSTM-AE 구조를 기반으로 선정

11.5 임계값(Threshold) 설정 및 탐지 로직

- 정상 검증 데이터의 재구성 오차 분포로부터

  → $\tau = \mu + k \cdot \sigma$ 또는 상위 99.5% 퍼센타일 기준으로 임계값 설정
- 실시간 Flow 입력 시 $\|X - \hat{X}\| \geq \tau$ → 비정상으로 판정

11.6 실시간 탐지 절차 (Real-time Inference Process)

(1) Flow Collector(Zeek/NetFlow)에서 Flow 로그 수집

(2) Raw Flow 저장 → 전처리

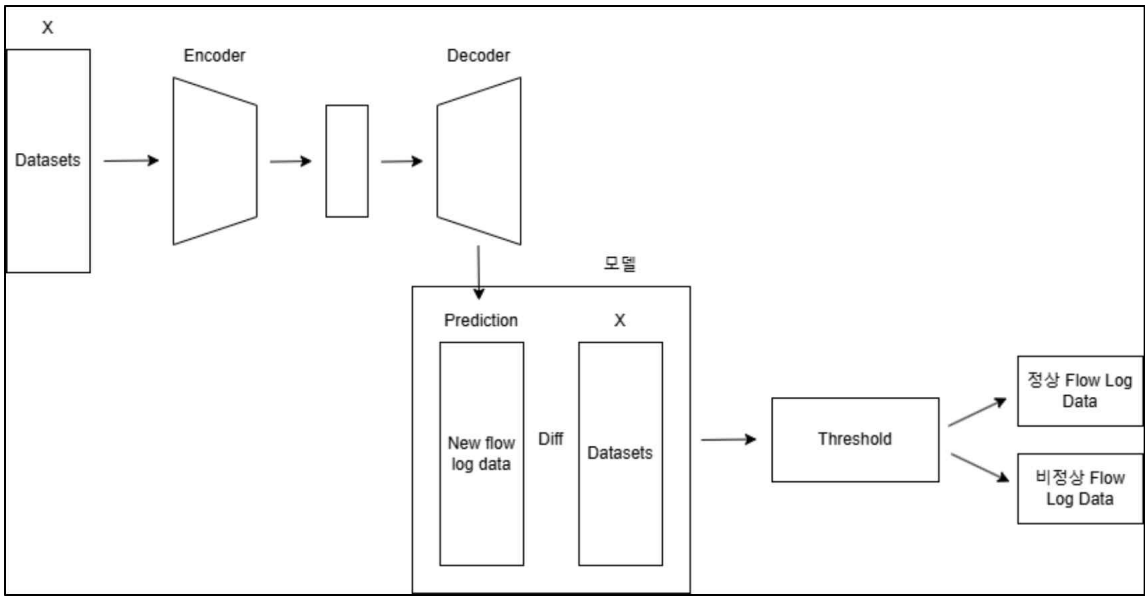
(3) 학습된 Autoencoder 모델 추론

(4) 재구성 오차 계산 → 임계값 비교

(5) 결과(정상/비정상) → Dashboard·CLI에 시각화 및 로그 저장

→ “실시간 탐지 단계 흐름도 Flow Chart” 참고

11.7 알고리즘 작동 예시 (Model Workflow)



(그림 11-1) Autoencoder 기반 Flow 이상 탐지 모델 구조도

입력: Datasets (Flow Log)

↓

Encoder → 잠재벡터 z 생성

↓

Decoder → 입력 재구성 ($\hat{x}$)

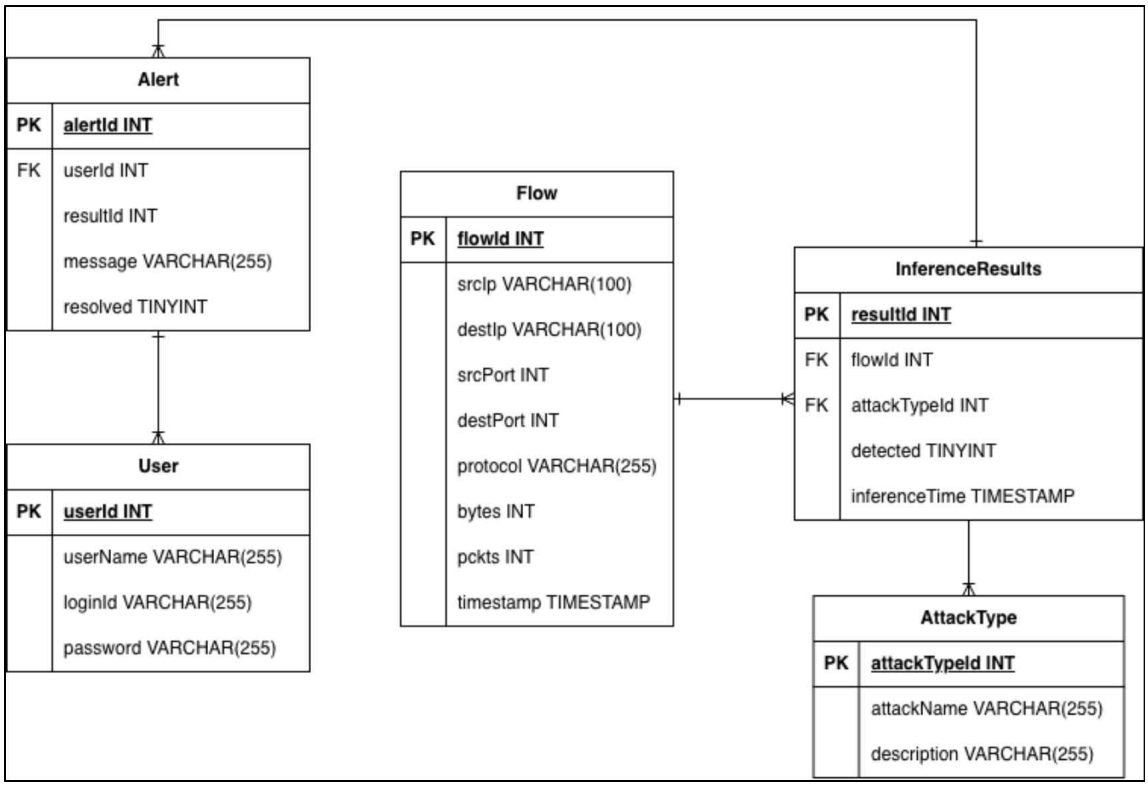
↓

오차 계산 $|x - \hat{x}|$

↓

Threshold 비교 → 정상 / 비정상 판정

12. 데이터베이스 테이블 구조 (DB Table)



13. 시나리오

1. 사용자는 회사 내부망에서 제공된 웹사이트 주소로 접속하여 로그인한다.

2. 로그인 후, 메인 화면에서 **실시간 네트워크 트래픽 현황**과 **최근 이상 탐지 결과**를 직관적으로 확인할 수 있다.

3. 사용자는 메뉴에서 탐지 설정을 선택해 "이상행위 탐지 시 관리자에게 실시간 알림", "자동 로그 기록" 등의 옵션을 직접 활성화해둔다.

4. [조건: 실시간 탐지 결과] 업무 시간 중 트래픽 패턴에 변화(특정 IP로부터 비정상적인 대량 접속 등)가 발생하면, AI 탐지 모델이 이를 자동으로 분석해 "비정상 트래픽 탐지" 배너와 팝업 알림을 띄운다.

5. 사용자는 팝업을 통해 상세 분석 리포트(공격 IP, 공격 유형, 탐지 시간

등)를 즉시 확인한다.

6. 관리자는 웹사이트 내 제공된 빠른 조치(예: 해당 IP 차단/모니터링) 버튼을 클릭하여 실시간으로 대응한다.

7. 복수의 알림이 동시 발생하거나 우선순위가 겹치는 이벤트가 있으면, 웹사이트는 중요도 기준 자동정렬, 가장 위험도가 높은 알림부터 상단에 배치하여 신속 대응을 가능하게 한다.

8. 모든 탐지 내역과 대응 기록은 웹사이트 내 로그 관리 메뉴에 자동 저장된다. 관리자는 여기서 전체 이력 및 분석 리포트를 다운로드해 내부 공유 혹은 추가 조사에 활용할 수 있다.

9. 사용자는 자신의 필요에 따라 알림 문구/민감도/자동화 규칙을 직접 설정하고, UI에서 각 기능을 편리하게 수정하면서 시스템을 운용한다.

14. 구현결과

14.1 OCSVM 모델 분석

14.1.1 학습 데이터 전략

OCSVM은 정상 데이터의 분포를 학습하여 결정경계를 생성하고 이 경계를 벗어나는 데이터를 이상으로 간주하는 방식이다. 모델이 공격패턴에 과적합되는 것을 방지하고 순수한 정상행위의 범위를 학습하기 위해, 전체 데이터셋 중 Benign(정상) 레이블을 가진 데이터만 추출하였다. 전체 정상 데이터중 연산 효율성과 일반화 성능을 고려하여 무작위로 100,000개의 샘플을 추출하여 학습데이터로 구성하였다. 학습에 사용되지 않은 정상데이터와 모든 공격 데이터는 테스트셋으로 분류하여 객관적인 성능평가를 수행하였다.

14.1.2 하이퍼 파라미터 튜닝

OCSVM의 성능은 커널의 종류와 주요 파라미터인 nu, gamma에 의해 크게 좌우된다. 최적의 파라미터 조합을 탐색하기 위해 GridSearch 기법을 적용하였다.

[주요 파라미터 설정]

- **Kernel:** 비선형 데이터 분포를 효과적으로 매핑하기 위해 **RBF** 커널을 고정적으로 사용하였다.
- **Nu :** 훈련 오차의 상한과 서포트 벡터의 하한을 조절하는 파라미터로, 이상치(Outlier)로 간주할 허용 오차 비율을 결정한다.
- **Gamma :** 결정 경계의 굴곡을 결정하는 파라미터로, 값이 클수록 복잡한 경계를, 작을수록 단순한 경계를 형성한다.

지정된 범위 내의 모든 조합에 대해 교차검증을 수행하였으며 F-1 Score를 기준으로 가장 높은 성능을 보이는 조합을 최종 파라미터로 선정하였다.

14.1.3 Threshold 정밀 탐색 및 최적화

OCSVM의 decision_function은 각 데이터 포인트가 결정경계로부터 얼마나 떨어져 있는지를 나타내는 score를 반환한다.기본적으로 음수이면 이상, 양수이면 정상으로 판별하지만 오탐과 미탐의 트레이드 오프를 조절할 필요가 있다.

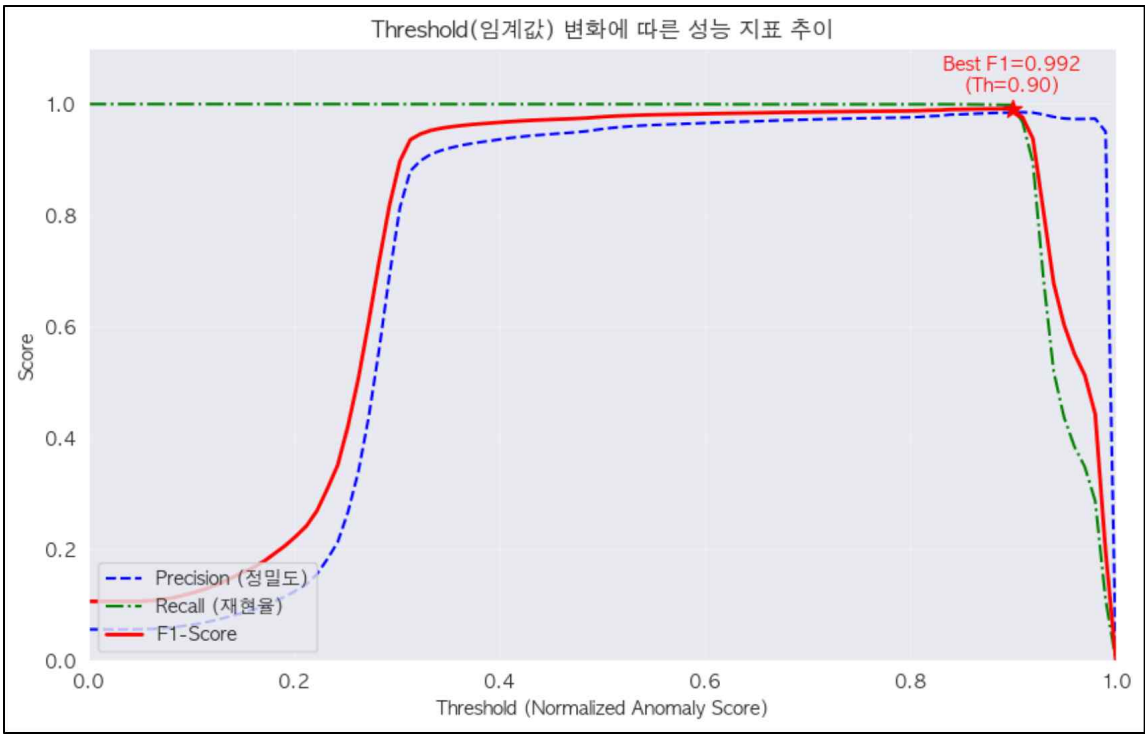
[탐색 알고리즘]

모델이 예측한 Score 값을 0과 1사이로 정규화하거나 Score 분포 범위를 기준으로 0.0부터 1.0사이의 임계값을 미세하게 조정하며 성능변화를 관찰하였다. Precisio과 Recall의 조화평균인 F1 Score가 최대가 되는 지적을 최적의 Threshold로 선정하였다.

[분석 결과]

0부터 1까지 threshold를 조정해나가면서 F1- Score가 0.9917로 최대가 되는 threshold 0.8990을 찾아냈다. 이때 정확도는 0.9849로 기본 threshold인 0보다

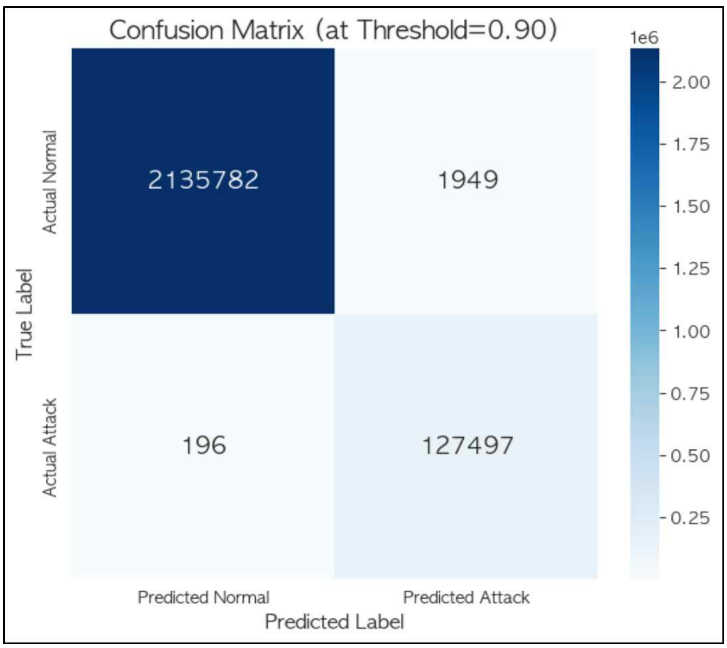
좋게 나왔다.



14.1.4. 최종 모델 성능 평가

위 과정을 통해 구축된 OCSVM모델을 테스트셋에 적용한 결과는 다음과 같다.

- Confusion Matrix

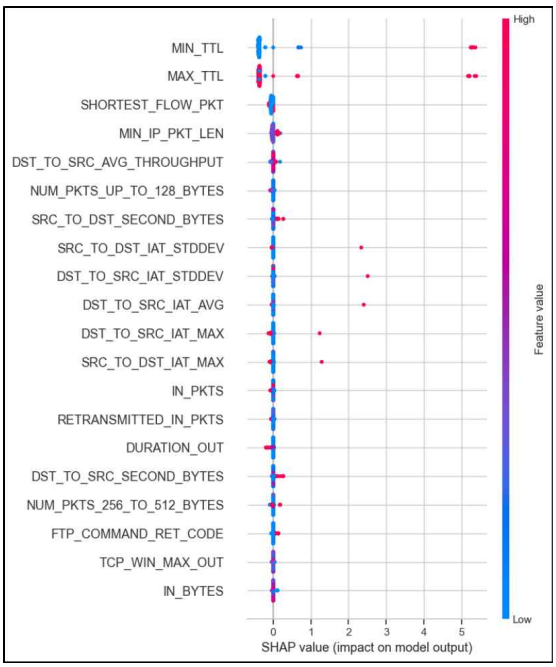


- 최종 성능 지표
  - Precision : 0.9849
  - Recall : 0.9985
  - F1-Score : 0.9917

14.1.6. SHAP기반 XAI 분석

OCSVM모델이 단순히 정상과 비정상을 구분하는 것을 넘어, 어떤 근거로 이상징후를 탐지했는지 알기 위해 Kernel SHAP 알고리즘을 적용하였다. 특히, OCSVM의 decision_function 결과에 음의 값을 취하여 SHAP값이 높을수록 공격에 기여하는것으로 정의하고 분석을 수행했다.

[전체 특징 중요도 분석]



위 그림에서 y축의 상단에 위치한 Feature들은 이상탐지에 가장 큰 영향력을 끼치는 변수들이다. 분석 결과 MIN_TTL과 MAX_TTL이 가장 중요한 요인으로 도출되었다. 붉은색 점이 오른쪽에 분포하는 경우 해당 Feature의 값이 클수록 모델이 이를 공격으로 판단할 확률이 높아짐을 의미한다.

14.2 TCN-AE 모델 분석

14.2.1 학습 데이터 전략

TCN 모델은 시계열 데이터를 입력으로 받아 시간적 패턴을 학습하는 신경망 기반의 이상 탐지 모델이다. 본 연구에서는 네트워크 트래픽의 시간적 흐름을 반영하기 위해 동일한 세션에 속한 레코드를 그룹화하여 입력 데이터를 구성하였다. 학습 및 평가(검증) 데이터셋은 Benign(정상)과 Attack(공격) 레이블을 모두 포함하도록 구성하였으며, 정상 행위와 공격 행위를 이진 분류(Binary Classification) 방식으로 모델을 학습하였다. 공격 행위는 총 9가지(Analysis, Backdoor, Dos, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, Worms)이다.

14.2.2 하이퍼 파라미터 튜닝

SessionTCN(TCN) 모델은 2-layer(1번째 블록: 32채널, 2번째 블록: 64채널) 구조로 구성하였으며, 지역적인 트래픽의 패턴을 학습하도록 설계하였다. 또한 네트워크 환경에서의 실시간 이상 탐지에 적합하도록 현재 시점 이전의 정보만 사용하도록 causal을 True로 설정하였다. 정상 행위와 공격 행위의 불균형성을 줄이기 위해 손실 함수에 pos_weight(10)을 적용하였다.

[주요 파라미터 설정]

1) 각 layer의 출력 채널 수(num_channel): [32,64]

2) 1D convolution 필터 크기(kernel size): 3

3) dropout: 0.2

4) causal: True

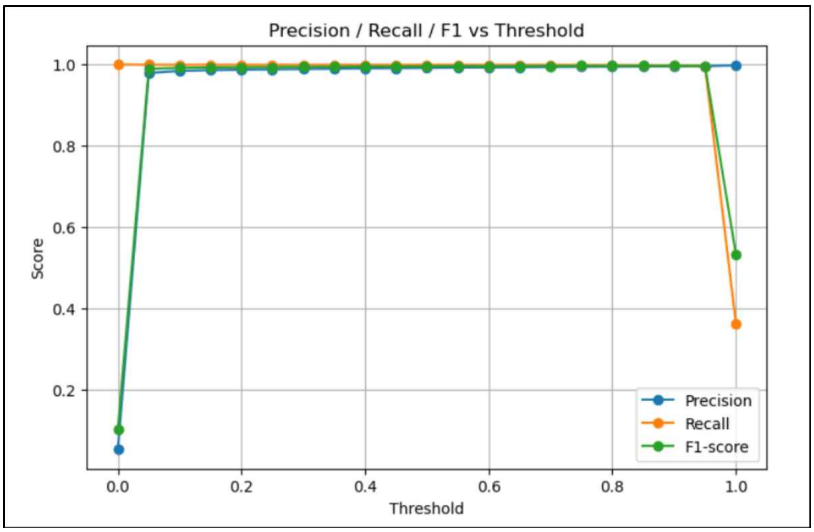
5) 정규화 방식(use_norm): 'weight_norm'

6) 활성화 함수(activation): 'relu'

7) 손실 함수(Loss Function): BCEWithLogitsLoss + pos_weight(10)

8) 옵티마이저(Optimizer): Adam(lr=1e-3)

14.2.3 Threshold 정밀 탐색 및 최적화



Threshold는 sklearn의 precision_recall_fscore_support 함수를 통해 precision, recall, f1-score 값을 계산한 뒤, 해당 지표를 기반으로 최적의 threshold를 선정하였다. 본 연구에서는 precision, recall이 모두 0.995 이상인 값 중 f1 score가 가장 높은 최적의 threshold로 최적 threshold(Best threshold = 0.881)를 선정하였다.

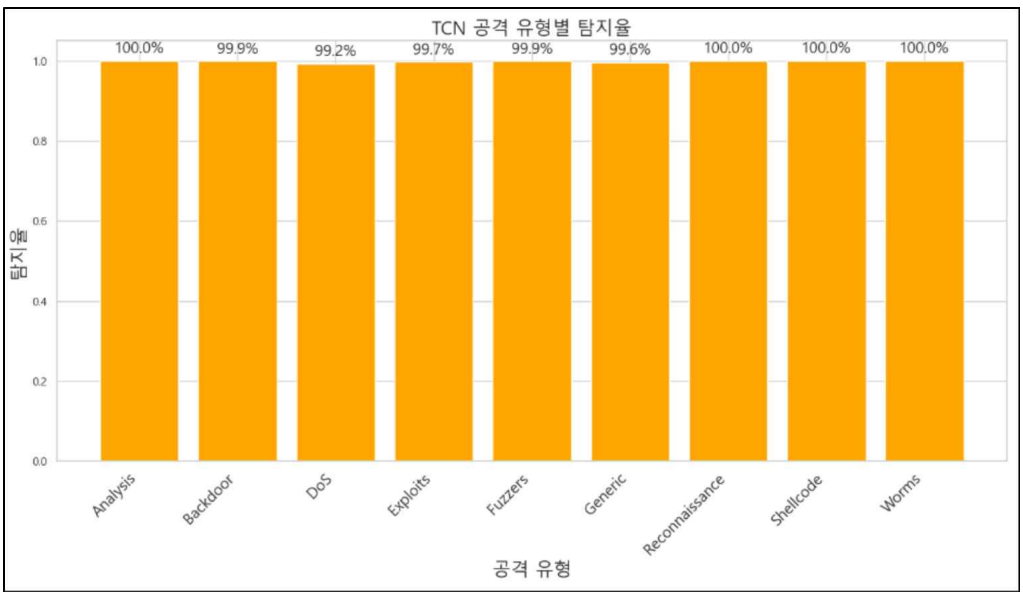
14.2.4. 최종 모델 성능 평가

14.2.4.1 최종 모델 Precision, Recall, F1-score

Metric	Value
Precision	0.9955
Recall	0.9981
F1-score	0.9968

최종적으로 학습시킨 모델은 threshold=0.881 기준 precision 0.9955, recall 0.9981, f1-score 0.9968의 결과를 보인다.

14.2.4.2 공격 유형별 탐지율



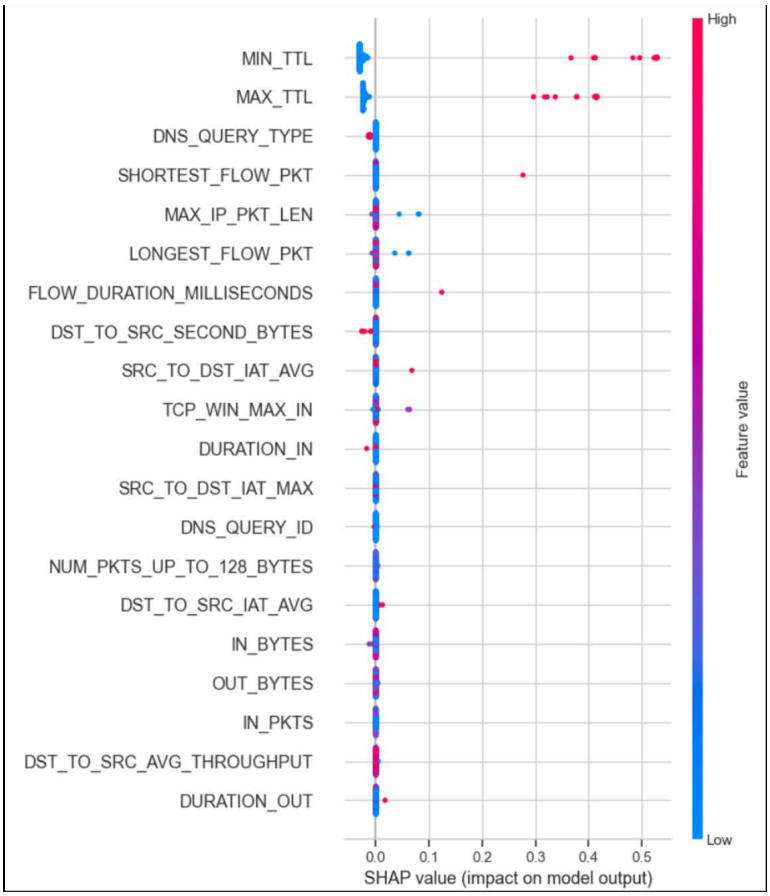

```
Analysis        | session_n=336 | Precision=0.0110  Recall=1.0000  F1=0.0217
BENIGN          | session_n=529627 | Precision=0.0045  Recall=0.0003  F1=0.0005
Backdoor        | session_n=1168 | Precision=0.0381  Recall=0.9991  F1=0.0734
DoS             | session_n=1552 | Precision=0.0503  Recall=0.9923  F1=0.0957
Exploits        | session_n=10555 | Precision=0.3436  Recall=0.9973  F1=0.5111
Fuzzers         | session_n=9220 | Precision=0.3008  Recall=0.9992  F1=0.4624
Generic         | session_n=1900 | Precision=0.0618  Recall=0.9963  F1=0.1164
Reconnaissance  | session_n=5064 | Precision=0.1653  Recall=0.9998  F1=0.2837
Shellcode       | session_n=708 | Precision=0.0231  Recall=1.0000  F1=0.0452
Worms           | session_n= 48 | Precision=0.0016  Recall=1.0000  F1=0.0031
```

공격 유형별 평가 결과, 모든 공격 클래스에서 Recall이 99% 이상으로 나타나, 모델이 공격 행위를 거의 놓치지 않고 탐지하고 있음을 확인할 수 있다. 반면 Precision 점수는 전반적으로 낮게 나타났으며, 이는 정상 트래픽을 공격으로 오탐하는 경우가 많은 것을 의미한다. 또한, 공격 클래스별 불균형에 의해 공격 유형별 precision 점수가 다른 것을 확인할 수 있다.

반면, BENIGN(정상) 클래스는 Recall이 0.0003으로 매우 낮은 값을 보인다. 이러한 결과는 본 모델이 공격 유형을 집중적으로 탐지하는 것에 최적화되어 있으나 정상 트래픽에 대한 오탐 비율이 높음을 의미한다.

14.2.5. SHAP기반 XAI분석

TCN 모델의 예측에 각 feature가 얼마나 기여했는지를 SHAP(SHapley Additive exPlanations)을 통해 분석하였다.



분석 결과 MIN_TTL과 MAX_TTL feature의 영향이 큰 것을 확인할 수 있다. TTL 값이 커질수록 공격으로 분류될 가능성이 증가하며, 반대로 TTL 값이 낮으면 정상 트래픽으로 판단될 가능성이 증가하는 것을 의미한다. 그 외의 feature들은 영향이 크다고 보기 어려우나, 일부 DNS_QUERY_TYPE 값이 큰 경우, 모델이 정상 행위로 판단하는 경향이 있다. 또한, 일부 SHORTEST_FLOW_PKT의 값이 큰 경우 공격 행위로 판단하는데 기여한다고 해석할 수 있다.

15. 역할 분담 및 개발일정

담당		개발구분	10월				11월				12월				1월				2월				3월				4월			
			2학기												(방학포함) 1학기															
			1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
설계	모두	관련 기술 및 논문 조사																												
	모두	기존 NIDS / Flow 기반 이상 탐지 분석																												
	모두	데이터셋 선정																												
	모두	제안서 보완 및 보충																												

구현	김○서	실험 환경 세팅 및 모델 실행 구조 설계																											
	송○용	데이터셋 정리 및 전처리 코드 작성																											
	안○엽, 이수아	실험 프로토타입 구조 설계																											
	김○서	프로토타입 통합 테스트 및 실행 흐름 점검																											

	송○용	Feature selection 및 기본 통계 분석																											
	안○엽, 이수아	머신러닝 베이스라인 모델 구현 및 학습																											
	김○서	실험 자동화 및 결과 저장 구조 구축																											
	송○용	결과 분석 및 데이터 불균형																											

		처리																											
	안○엽, 이수아	딥러닝 모델 구현																											
	김○서	전체 실험 결과 및 성능 비교 시각화																											
	송○용	데이터 및 실험 결과 문서화																											
	안○엽, 이수아	모델 분석 및 비교 문서화																											

	김○서	서버 구조 설계 및 DB설계 기본 API 구현																											
	송○용	서비스화면 기획 및 와이어프레임 제작																											
	안○엽	기존 모델 리팩토링 및 추론 속도 개선																											

	김○서	모델 연동 및 파이프라인 연결																											
	송○용	모델 결과 시각화 기능																											
	안○엽	Fine-tuning 및 정확도 향상																											
	모두	통합 테스트 및 서비스 배포																											
정리	모두	보고서 정리																											
	모두	최종 마무리																											

16. 관련기술 및 개념

- Flow 데이터 분석 기술

아래의 데이터셋을 전체 공통 전처리를 진행한다.

- 머신러닝/딥러닝 알고리즘

1. LSTM-Autoencoder

이유: 시퀀스 재구성오차로 이상을 잡음(저속 C2·주기성에 강함). 논문 결과와도 부합. ㅎㅎ

2. TCN-Autoencoder (Temporal Convolutional Network)

이유: 병렬 처리로 학습/추론 빠름. 장기 의존성 포착 능력 좋음. 경량화 시 LSTM보다 효율적일 때가 많음.

3. Seq-Transformer (경량화 버전) 또는 LSTM + Attention (작은 Transformer 블록)

이유: 병렬 처리로 빠르며, 장기 의존성 포착 능력 우수. 경량화에 유리.

→ Seq-AE(1D-CNN Encoder + LSTM Decoder)

→ 딥러닝 3종 대신 한 모델(예: LSTM-AE) + 비딥(예: IsolationForest) 을 포함해 앙상블 성능을 보는 것도 현실적(논문은 I-Forest/AE/SVM 비교 결과 제시). ㅎㅎ

- 데이터셋

1. https://staff.itee.uq.edu.au/marius/NIDS_datasets _ NetFlow Datasets for ML-based NIDS (NF-UNSW-NB15, NF-BoT-IoT 등)

2. https://zenodo.org/records/6610489 _ AIT Netflow Dataset

3. https://repo.researchdata.hu/dataset.xhtml?persistentId=hdl%3A21.15109%2FARP%2FFBIIOZ _ Anonymized Netflow / security scan data (Hungary ARP repository)

4. https://www.mdpi.com/2306-5729/8/1/18 _ UWF-ZeekData22 / UWF-ZeekDataFall22

5. https://www.kaggle.com/datasets/mimansari/dns-and-connections-log-zeek-or-corelight _ Kaggle: DNS + Connections log (Zeek/Corelight)

6. https://www.unb.ca/cic/datasets/ids-2017.html _ CIC-IDS2017

7. https://research.unsw.edu.au/projects/unsw-nb15-dataset _ UNSW-NB15

8. https://www.caida.org/catalog/datasets/about/downloads/ _ CAIDA 데이터셋

9. https://registry.opendata.aws/cse-cic-ids2018/ _ CSE-CIC-IDS2018

(a.k.a. CIC-IDS2018)

17. 기대효과

- 실시간 이상행위 탐지: 새로운 공격 유형에도 대응 가능한 AI 기반 탐지 시스템 구축
- 운영 효율성 향상: 패킷 기반 분석 대비 계산 비용 절감 및 빠른 처리
- 확장성 확보: 클라우드 환경, IoT 네트워크, 기업망 등 다양한 환경에 적용 가능
- 교육 및 연구 활용: Flow 기반 AI 탐지 연구 및 실무 실습 데이터로 활용 가능

활용 예)

- 사용자는 우리 모델 다운 → 모델이 사용자의 컴퓨터에 있으면, 이 모델이 네트워크 flow 데이터 소스를 분석해서 학습해둔 데이터에서 벗어나는 이상치가 존재할 시, 네트워크 공격 감지 → 사용자에게 네트워크 공격 탐지 되었다고 출력

18. 개발도구 및 개발환경

- 개발도구 : tensorflow keras, pytorch - python 3.10

- 개발환경 : google colab

19. 참고문헌

-https://manuscriptlink-society-file.s3-ap-northeast-1.amazonaws.com/kics/conference/koreaai2021/presentation/0046.pdf (네트워크 플로우 데이터 기반 이상징후 탐지 인공지능 모델 성능 비교)

- https://www.earticle.net/Article/A403316 (머신러닝과 딥러닝을 활용한 악성 패킷 탐지 기술 연구)

- https://journalofbigdata.springeropen.com/articles/10.1186/s40537-024-00886-w (Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction)
- https://www.mdpi.com/2078-2489/16/7/515 (Impact of Machine Learning on Intrusion Detection Systems for the Protection of Critical Infrastructure)
-https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741/full (Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability)
- https://www.mdpi.com/1999-4893/15/12/453 (Packet-Level and Flow-Level Network Intrusion Detection Based on Reinforcement Learning and Adversarial Training)
- https://www.mdpi.com/1424-8220/23/5/2415 (Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure)
- https://arxiv.org/abs/1007.1266 (Network Anomaly Detection: Flow-based or Packet-based Approach?)
- https://www.sciencedirect.com/science/article/abs/pii/S0167404817301165 (Flow-based intrusion detection: Techniques and challenges)
- https://scholar.smu.edu/datasciencereview/vol3/iss3/8 (Flow-Based and Packet-Based Intrusion Detection Using BLSTM)
  - https://ris.utwente.nl/ws/files/6523249/Surveys_and_tutorial.pdf (An Overview of IP Flow-Based Intrusion Detection)
  - https://arxiv.org/abs/2011.09144 (NetFlow Datasets for Machine Learning-based Network Intrusion Detection Systems)
  - https://arxiv.org/abs/2102.09980 (A flow-based IDS using Machine Learning in eBPF)
  - https://www.sciencedirect.com/science/article/pii/S2665917423001630 (A comprehensive review of AI based intrusion detection system)